AI ENGINEER · DATA · WEEK 3, DAY 5

Giving the model tools

Function calling, the tool loop, OpenAPI and MCP, then a tour of Azure AI Foundry

4 hours · instructor-led · hands-on lab

Where we are in Priya's story

- Priya can now find the refund policy and quote it with a citation
- The policy says eligibility depends on how many claims the customer has already made this month
- That number is not in any document. It is in the warehouse
- No amount of better retrieval will find it, because there is nothing to retrieve

The shift this week

Weeks 1 and 2 searched unstructured text.

This week the answer has to be computed from structured data, which is a different problem with different failure modes.

Most real enterprise questions need both halves. That is why they are hard, and why most assistants only do one.

Six questions before we start

FINDING THE RIGHT CHUNK

1. Why run keyword search and vector search together?
2. What does a reranker do that search cannot?
3. Search puts a withdrawn policy first. What fixes it?

TRUSTING THE ANSWER

4. What does nDCG@10 tell you that “we found it” does not?
5. The answer cites a chunk. How do you know the citation is real?
6. When should Priya’s assistant refuse to answer?

IN CHAT Pick one number and answer it in a sentence. Two minutes, then we compare.

Six questions before we start: the answers

FINDING THE RIGHT CHUNK

1 Why run keyword search and vector search together?

Each finds what the other misses. Keywords match exact terms such as a policy code; vectors match meaning. We merged the two lists by rank position.

2 What does a reranker do that search cannot?

It reads the question and each shortlisted chunk together, and scores whether the chunk answers it. Search only compares how alike they look.

3 Search puts a withdrawn policy first. What fixes it?

A metadata filter. A withdrawn policy reads exactly like a current one, so similarity cannot tell them apart. A status label can.

TRUSTING THE ANSWER

4 What does nDCG@10 tell you that "we found it" does not?

Whether the right chunks came near the top, scored 0 to 1. Our three changes took it from 0.803 to 0.874 offline, and 0.862 to 0.987 with real models.

5 The answer cites a chunk. How do you know the citation is real?

Check that the cited chunk was retrieved and holds the quoted words. In Lab 4, 8 of 8 fake citations were caught before reaching the user.

6 When should Priya's assistant refuse to answer?

When the documents do not hold the answer. On 5 unanswerable questions the offline writer refused 0; a real model refused 4.

TODAY Same habits, new kind of answer: Priya's number is not in any document. It has to be computed from the warehouse.

Today in four stops



Tools

how a model asks your code to act

Sharing tools

OpenAPI and MCP

Foundry tour

45 minutes, live in the portal

Lab 5

your own function, two ways in

By the end: your own function as an agent's tool, served two ways, and an agent that hands its answer on.

1

Tools

How a model asks your code to do something

A model on its own only knows what it read in training

WITHOUT TOOLS

"Where is my parcel?"

"I do not have access to your orders. Please check the shop's website."

or worse: it guesses, and sounds sure

WITH A TOOL

"Where is my parcel?"

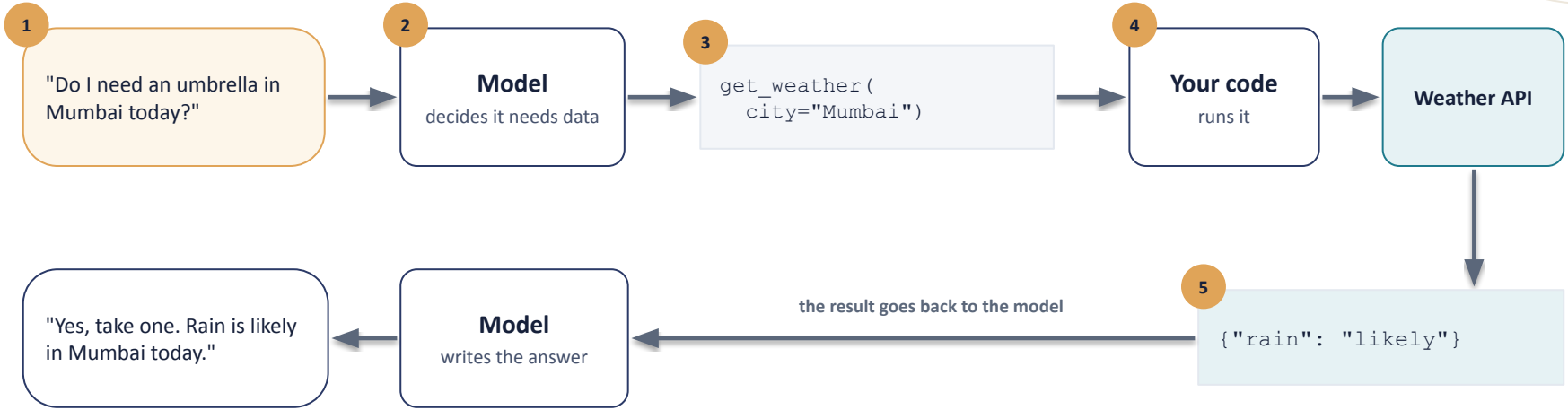
```
track_order(customer="priya")
```

"It left the warehouse this morning and arrives tomorrow."

the answer comes from the shop's own system

Live, private or company data needs a tool. The model cannot know it.

One tool call, start to finish



Function calling, tool calling and tool use are three names for this one idea.

The model asks. Your code acts.

What the model sees of a tool

YOUR PYTHON FUNCTION

```
@tool
def get_weather(city: str):
    """Today's forecast for one city."""
    key = load_secret()
    reply = call_weather_api(city, key)
    return reply.summary
```

WHAT THE MODEL RECEIVES

NAME

`get_weather`

DESCRIPTION

Today's forecast for one city.

INPUTS

`city: text`

Three things go to the model. The code inside, grey on the left, never does.

The same idea, in four real products

AIRLINE APP

"Find me a flight to Delhi on Friday"

```
search_flights(
  to="DEL", day="Friday")
```

BANK CHATBOT

"I lost my card, block it now"

```
block_card(
  card="debit")
```

IT HELPDESK

"I am locked out of my laptop"

```
reset_password(
  user="priya")
```

ONLINE SHOP

"Where is my parcel?"

```
track_order(
  customer="priya")
```

A sentence in. A tool name and inputs out. Your code does the rest.

Your code decides what runs

RUN IT

only reads

```
get_weather(...)
```

```
track_order(...)
```

```
search_flights(...)
```

ASK THE USER FIRST

changes something for this user

```
block_card(...)
```

```
cancel_booking(...)
```

```
reset_password(...)
```

A PERSON APPROVES

moves money or deletes data

```
transfer_money(...)
```

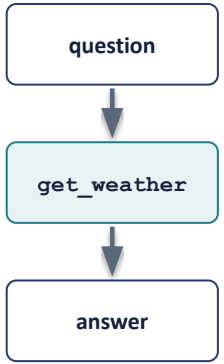
```
issue_refund(...)
```

```
delete_records(...)
```

Put the lanes in code, never only in the prompt. Lab 5 shows how.

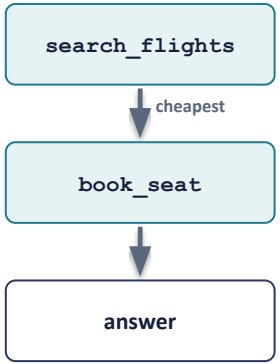
Four ways to combine tool calls

1 One call



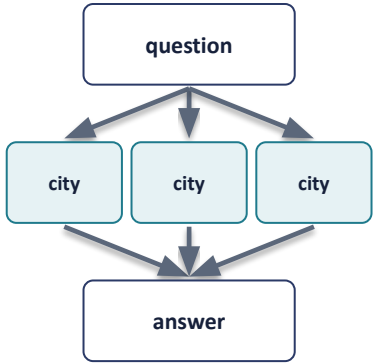
"Weather in Mumbai?"

2 A chain



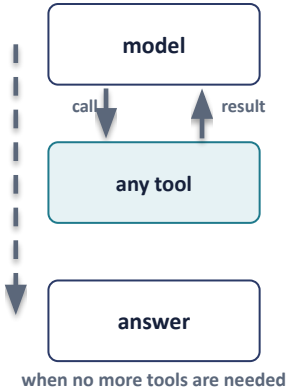
"Book the cheapest flight"

3 In parallel



"Weather in three cities"

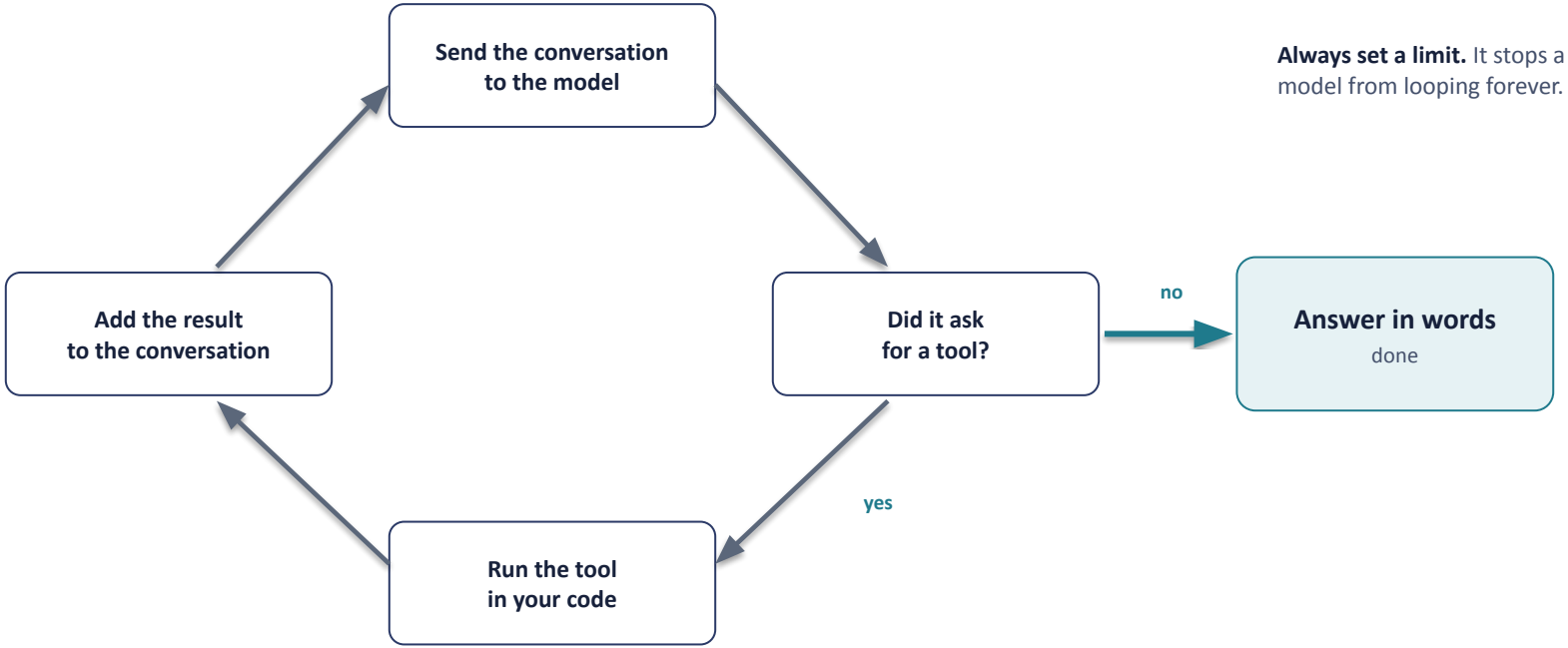
4 A loop



"Plan my weekend trip"

Pattern 4 can do all the others. It is the start of an agent: Week 4.

The tool loop: the code behind pattern 4



Always set a limit. It stops a confused model from looping forever.

Foundry's agents run this loop for you. In Lab 5 one question makes two calls.

Break

10 minutes. After the break: how one tool can serve many apps.

2

Sharing tools

OpenAPI and MCP: describe a tool once, use it everywhere

An API that describes itself can become tools

A SHOP'S API, DESCRIBED IN OPENAPI

```
paths:
  /orders/{id}:
    get:
      summary: Get one order
  /orders:
    post:
      summary: Place an order
  /refunds:
    post:
      summary: Start a refund
```

converted

get_order

Get one order

place_order

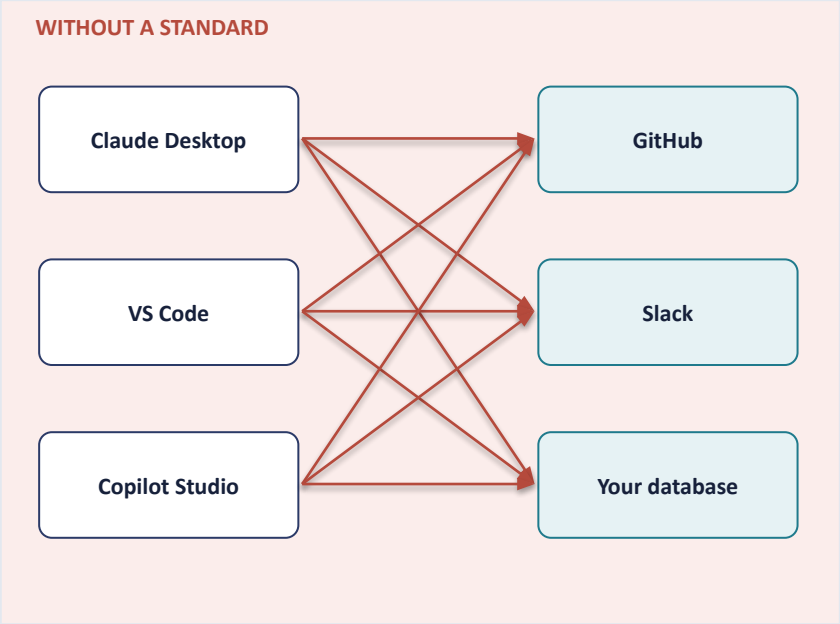
Place an order

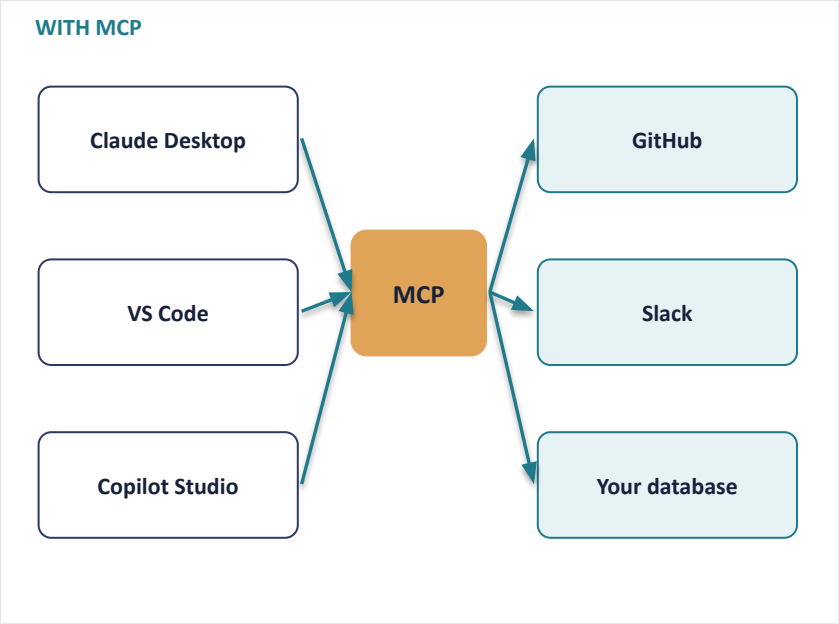
start_refund

Start a refund

No new tool code. Azure AI Foundry agents can take an OpenAPI file as a tool.

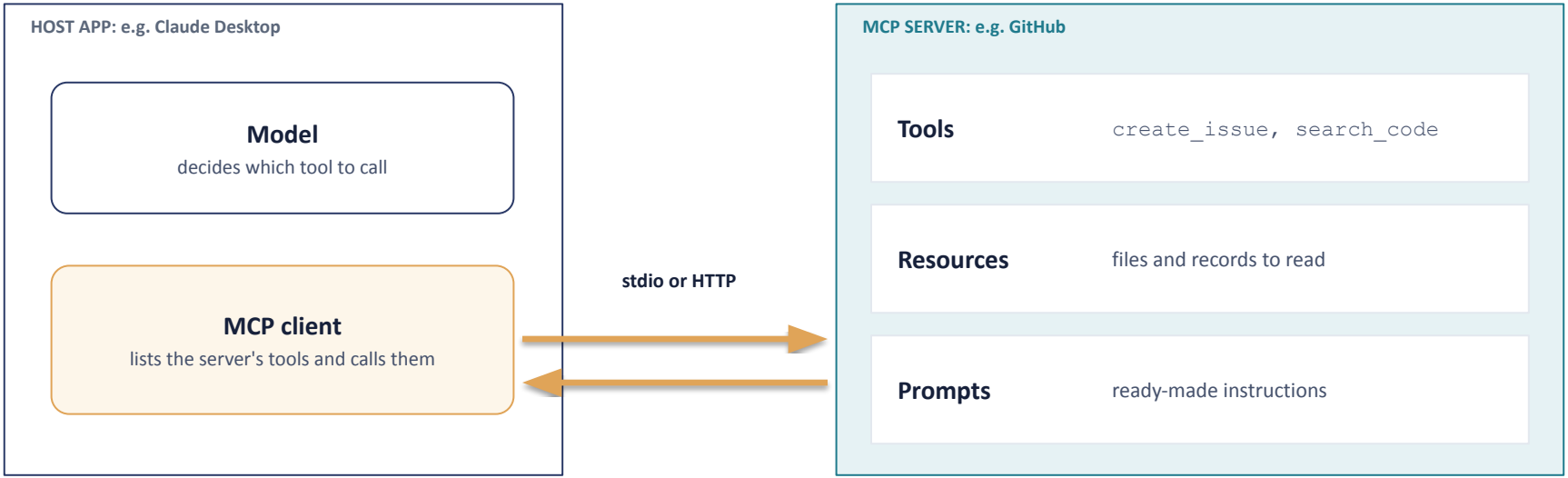
Every app wiring every tool by hand does not scale





Write the tool once, as an MCP server. Every MCP app can use it.

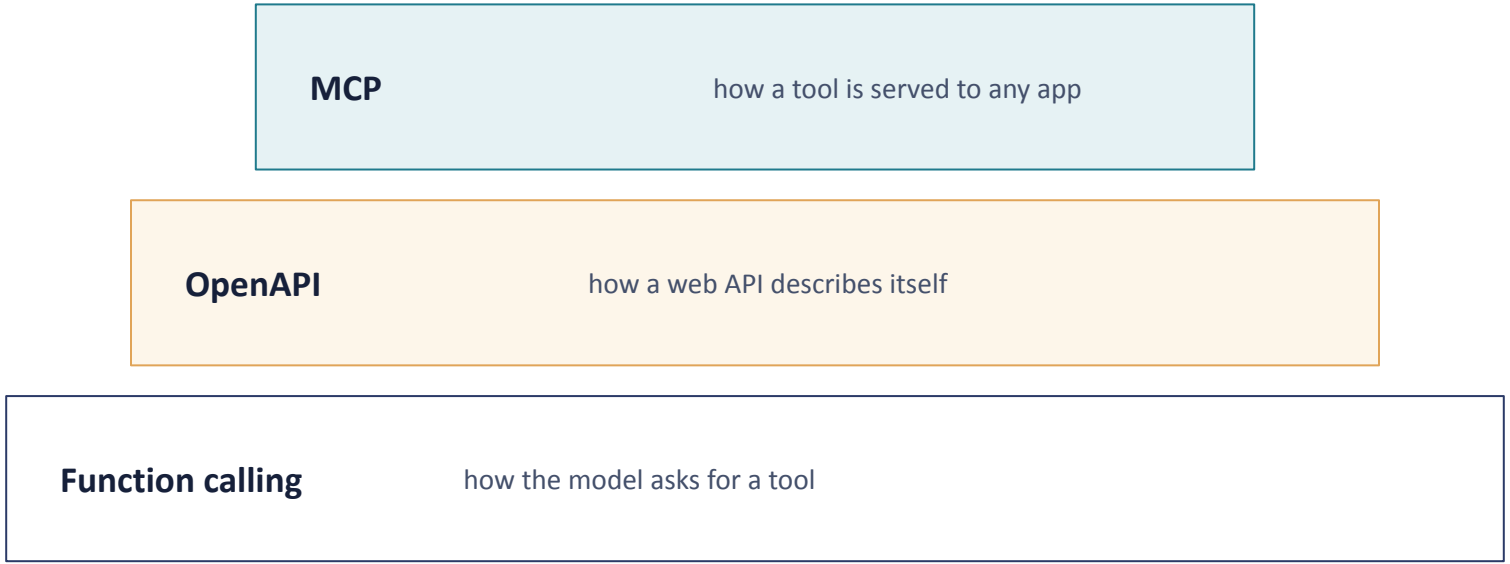
The parts of MCP



stdio: the server runs on the same machine and talks through its input and output. HTTP: the server runs elsewhere.

Lab 5 puts its own tools in an MCP server and gets the same answer.

Three names, three layers



They stack. A model calls tools; OpenAPI and MCP decide where the tools come from.

QUICK CHECK

Three questions before the lab

1

A bank bot has the tool transfer_money. Which lane does it go in, and where is that rule written?

2

You want three AI apps to use one internal tool. What do you build?

3

The model keeps choosing the wrong tool. What do you change first?

Answers in chat. Wait for two people to commit before the next slide.

ANSWERS

Three questions before the lab

1

A person approves

It moves money. The rule lives in your code, which refuses to run it without an approval. Never only in the prompt.

2

An MCP server

Write the tool once as a server. Each app connects to it and gets the same code and rules.

3

The description

The model chooses from the name, description and inputs alone. Make them say when to use the tool.

3

Azure AI Foundry tour

Where models live on Azure, and how code reaches them. 45 minutes, live in the portal

Three stops in Foundry



Model catalog

Browse models from Microsoft, OpenAI, Meta, Mistral and others. Read a model card.

Playground vs API

Try a prompt by hand. Then View code: the same call from Python.

Deploy a model

A deployment gives a model a name and an endpoint your code calls.

Lab 5 builds its agents on one of these deployments: gpt-4.1-mini.

Break

10 minutes. After the break: Lab 5, built in Azure AI Foundry.

4

Lab 5 on FreshCart

The same ideas in Microsoft Foundry: a document, an MCP server, your own function, and a second agent

Lab 5

Give the model tools, and check what it does with them

About 50 minutes · Foundry playground and the Azure portal · Lab 5 handbook

1. Ground it on the refund policy
2. Add Microsoft's MCP server
3. Your function in Azure
4. Your function as an OpenAPI tool
5. The same function as an MCP server
6. Which tool does it pick?
7. A second agent

How Lab 5 is wired

IN FOUNDRY

FirstAgent

gpt-4.1-mini

File search: the refund policy
Microsoft Learn MCP server
freshcart_metrics (OpenAPI)
freshcart-mcp (MCP)

FUNCTION APP

/api/get_metric

a web address, for the OpenAPI tool

/runtime/webhooks/mcp

the same lookup, as an MCP server

ONE SHARED LOOKUP

lookup()

each official number written once
freshcart.db, opened read-only

Then Replywriter, a second agent, turns the answer into a customer email.

One agent, four tools: a document, Microsoft's MCP server, and your own function two ways.

LAB 5, PART 1

Grounded on one document, it cites or it refuses

"How many refund claims can a customer make in thirty days?"	Three in any rolling thirty-day period, cited
"How long after delivery can a customer make a claim?"	48 hours after delivery, cited
"What is the refund policy for grocery orders in Singapore?"	Refuses, or quotes the Scope section: India
"What was the refund rate in Q1 2024?"	Refuses: that number is in a document it was not given

Remove the file and the "only" instruction, ask about Singapore again: a confident general answer about policies FreshCart never wrote.

The same question, with and without grounding, makes the point in one minute.

What Lab 5 found

Trainer runs, 14 and 15 October 2026, on Foundry with gpt-4.1-mini. Compare with your own run.

One function, two ways in, the same number

OpenAPI tool

tool freshcart_metrics
runs straight away, no approval step



/api/get_metric



531,043.03

revenue, March 2024

MCP tool

tool freshcart-mcp
shows Approve and Deny before it runs



/runtime/webhooks/mcp



531,043.03

revenue, March 2024

The model cannot tell where a tool runs. That is the point.

The name is the contract: two tools called get_metric

freshcart_metrics

function: get_metric

freshcart-mcp

function: get_metric

ASKED

"What was revenue in March 2024? Use the freshcart_metrics tool."

WHAT HAPPENED

It still called the MCP tool. The model never sees the portal's labels.

THE FIX

Give the OpenAPI function its own name: operationId get_metric_openapi.

Then steer with one instruction line.

The model sees function names and descriptions, not the labels in the portal.

Which key opens what: the most common error

Key	OpenAPI web address	MCP address	Where to find it
default host key	opens it	403, not allowed	App keys, Host keys
mcp_extension key	not this one	opens it	App keys, System keys
no key, or a wrong one	401, not signed in	401, not signed in	

Foundry's 401 means: wrong key, or no key at all. Check the error: if the call has no x-functions-key line, the tool's authentication is set to Anonymous.

Most Lab 5 tool errors were the wrong key, not wrong code.

The model asks, Foundry asks you

1

The agent decides to call get_metric

with revenue and 2024-03

2

Foundry pauses and shows the call

mcp_list_tools, then mcp_approval_request

3

You click Approve or Deny

Approve once, for the demo

4

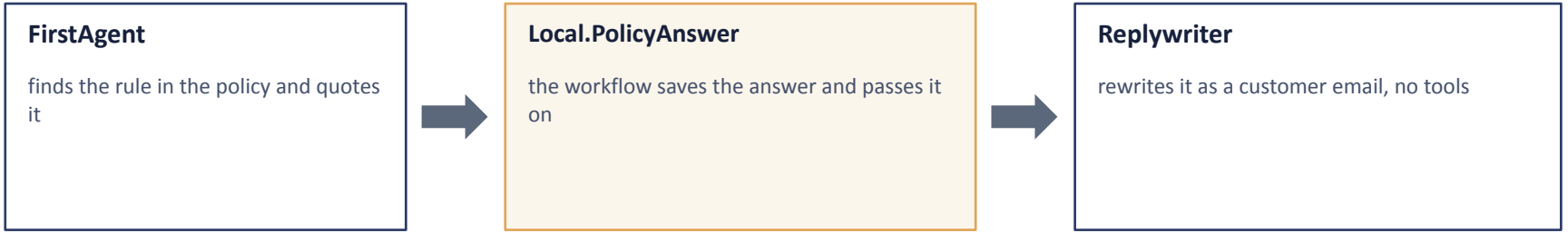
The answer, with its tool named

$531,043.03, from freshcart-mcp

This is the amber lane from slide 11, built into the platform: MCP tools ask first. The OpenAPI tool ran straight away, so what it can do must already be safe: get_metric only reads.

Approval is a lane in the platform. What the function can do is a lane in the code.

A second agent: where made-up facts slip back in



WITHOUT THE HANDOVER

Replywriter got an empty message and replied "0 ->".

WITH A LOOSER INSTRUCTION

"The policy applies to India" became "refunds for Singapore are not authorized": a decision the policy never makes.

The fix was in the instructions of both agents. Click Traces to see what each one received.

AI ENGINEER · DATA · WEEK 3, DAY 6

Getting the number

Turning a plain-English question into a number you can defend: NL2SQL, a semantic layer, narration and charts

4 hours · instructor-led · hands-on lab

Yesterday in one picture

The model asks, your code acts

Foundry asked for approval before every MCP call

The name is the contract

Two tools named get_metric: it could not tell them apart

Official numbers need an official tool

531,043.03 from get_metric, over OpenAPI and MCP

Today: when there is no official tool, how does the model's own SQL get the right number?

Today in four stops



Question to SQL

NL2SQL and why it is hard

Grounding

schema, rules, examples, checks

Semantic layer

one definition per number

Narration and charts

the last step, told right

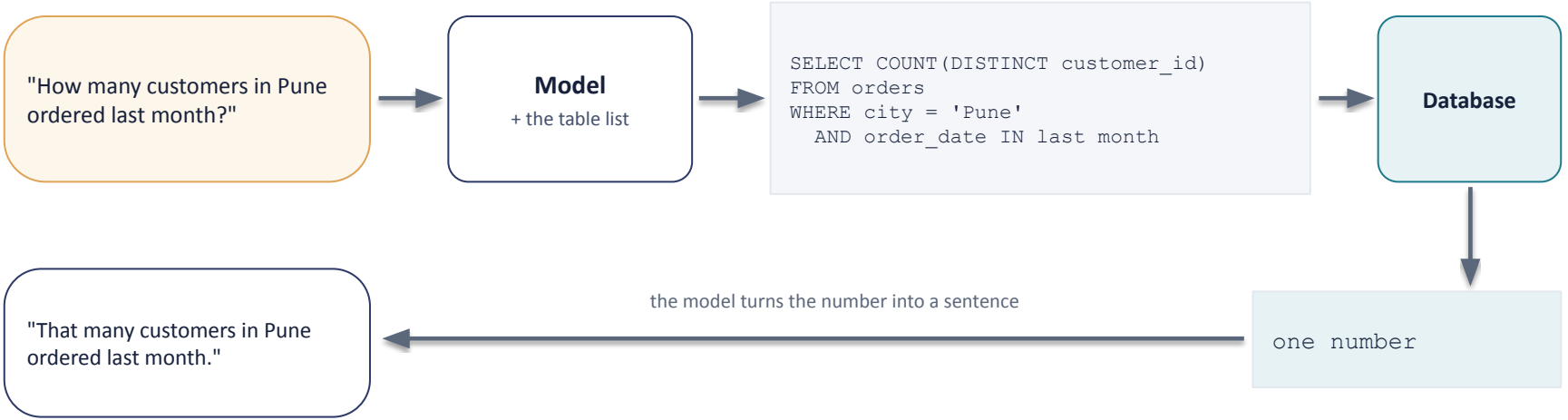
Then Lab 6 does all four on FreshCart's data.

1

Question to SQL

NL2SQL: a question in English, a query for the database

NL2SQL in one picture



Writing the SQL is the easy part. Knowing what the words mean is the hard part.

Where teams use it

SALES HEAD

"Our best-selling products this quarter?"

```
GROUP BY product
ORDER BY units DESC
```

HR LEAD

"How many people joined in March?"

```
COUNT(*) FROM employees
WHERE join_month =
'March'
```

SUPPORT MANAGER

"Which city has the most complaints?"

```
GROUP BY city
ORDER BY COUNT(*) DESC
```

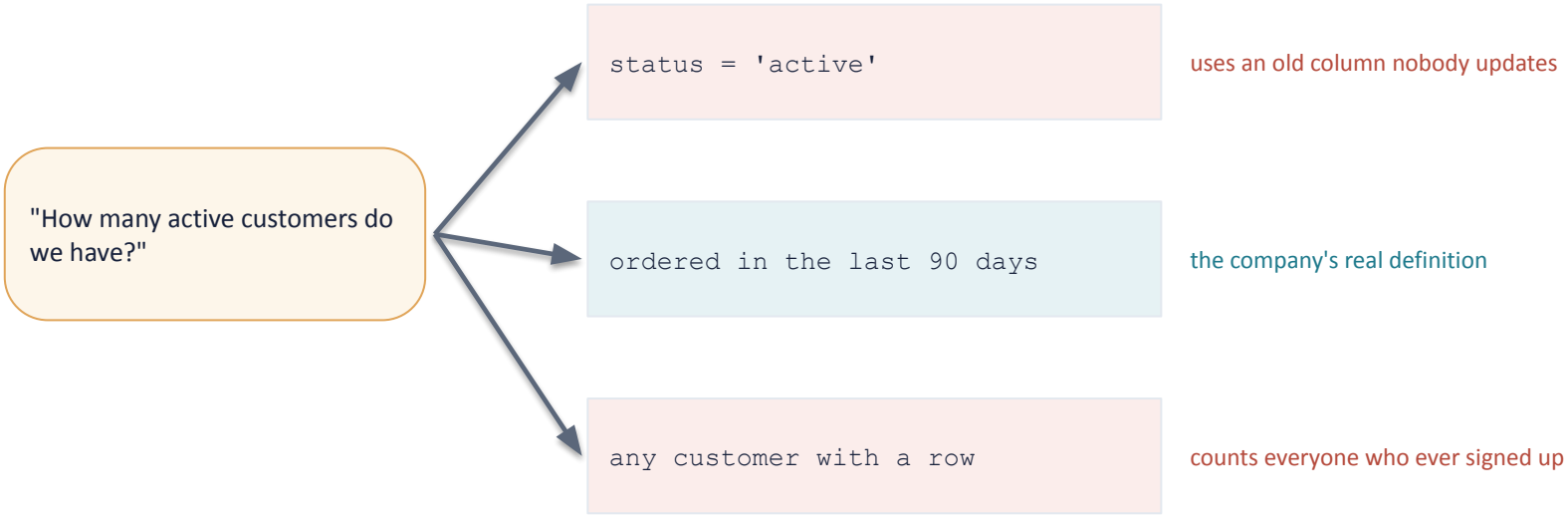
FINANCE ANALYST

"Revenue by region this year?"

```
SUM(revenue)
GROUP BY region
```

People who cannot write SQL get answers without waiting for an analyst.

One question, three reasonable queries



All three run without an error. Only one matches what the business means.

Six ways a query is wrong without an error

Wrong table

an old copy still looks real

Wrong definition

"active" means something specific here

Wrong grain

adding up month-end totals

Wrong join

one order counted once per item

Missing filter

test accounts left in

Ambiguous time

"last month" from which date?

Each one returns a number that looks exactly like a right one.

2

Grounding

Hand the model what it could not possibly know, then check its work

Three things to send with every question

1

The schema

tables, columns, types

```
orders_v2(order_id, customer_id,
  order_date, order_value, status)
```

2

Business rules

what the words mean

```
- exclude test accounts
- revenue = delivered orders only
```

3

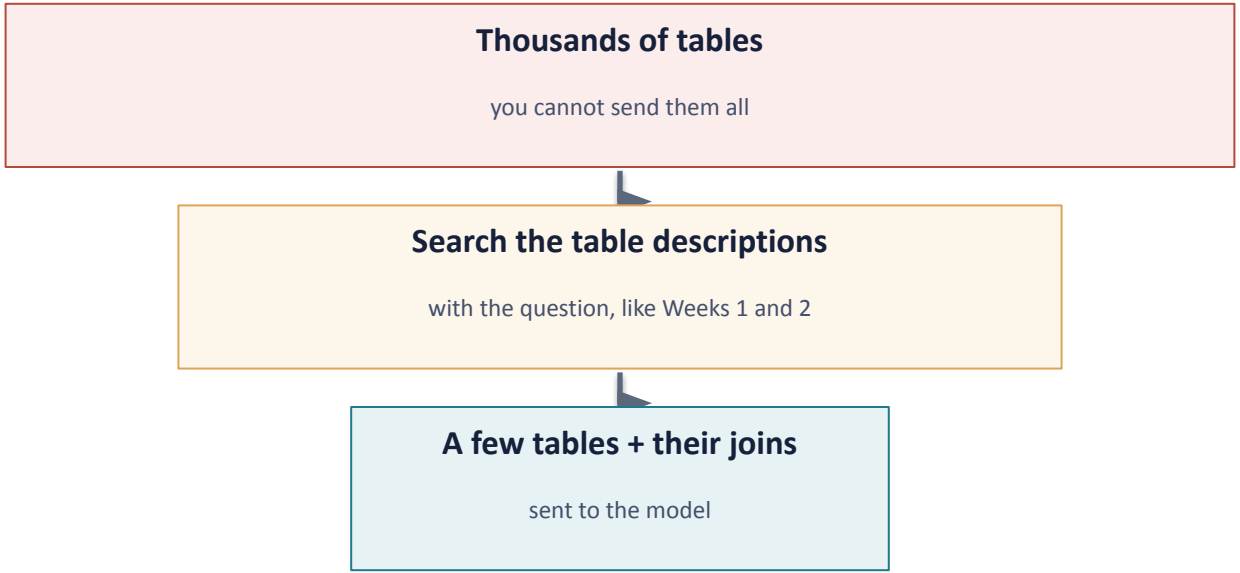
Worked examples

real questions with reviewed SQL

```
Q: Orders in March?
SQL: SELECT COUNT(*) ...
```

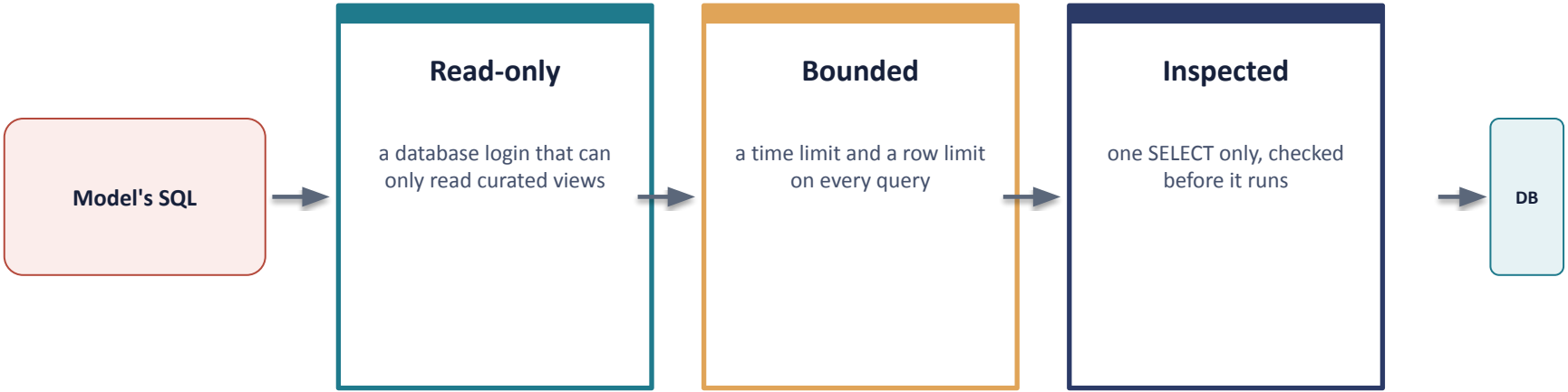
The schema comes free. Rules and examples are work, and they are where accuracy is won.

A real warehouse has thousands of tables. Send only a few



Finding the right tables is retrieval, the skill from Weeks 1 and 2.

Generated SQL is untrusted input: three gates

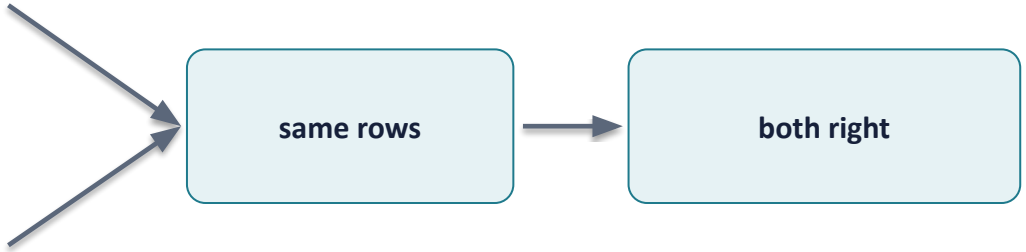


Yesterday's lesson again: the protection lives in code, not in the prompt.

Right means the same rows, not the same SQL

```
SELECT COUNT(*) FROM orders o
JOIN customers c ON ...
WHERE c.is_test = 0
```

```
SELECT COUNT(*) FROM orders
WHERE customer_id NOT IN
  (SELECT id FROM test_accounts)
```



Exact match compares the SQL text and would mark one of these wrong. **Execution accuracy** runs both and compares the answers.

Score on a set of real questions with reviewed answers. Lab 6 keeps an answer key of reviewed queries.

Sometimes the right answer is a question back

Ambiguous

"Show me top customers"

↓

"By revenue, by orders, or most recent?"

Not in the data

"Why did customers leave?"

↓

"The data has no reason for leaving."

Unclear dates

"Orders last month"

↓

"Which month do you mean?"

A refusal is a success. A confident wrong number is the failure.

QUICK CHECK

Three questions before the break

1

Your NL2SQL bot writes valid SQL every time and is still often wrong. Where do you look first?

2

Two queries look different but return the same rows. Is one of them wrong?

3

Name the three gates between generated SQL and your database.

Answers in chat. Wait for two people to commit before the next slide.

ANSWERS

Three questions before the break

1

What you send it

Missing business rules and examples, not the model. The meaning is the hard part.

2

No

Execution accuracy compares results. Same rows means both are right.

3

Read-only, bounded, inspected

A read-only login, time and row limits, and one SELECT only, checked in code.

Break

10 minutes. After the break: writing each definition once, for every tool.

3

Semantic layer

Write each business number once, and let every tool ask that place

Three teams, one word, three numbers

Finance

"Revenue last quarter"

```
revenue after refunds
```

Sales

"Revenue last quarter"

```
bookings when signed
```

Product

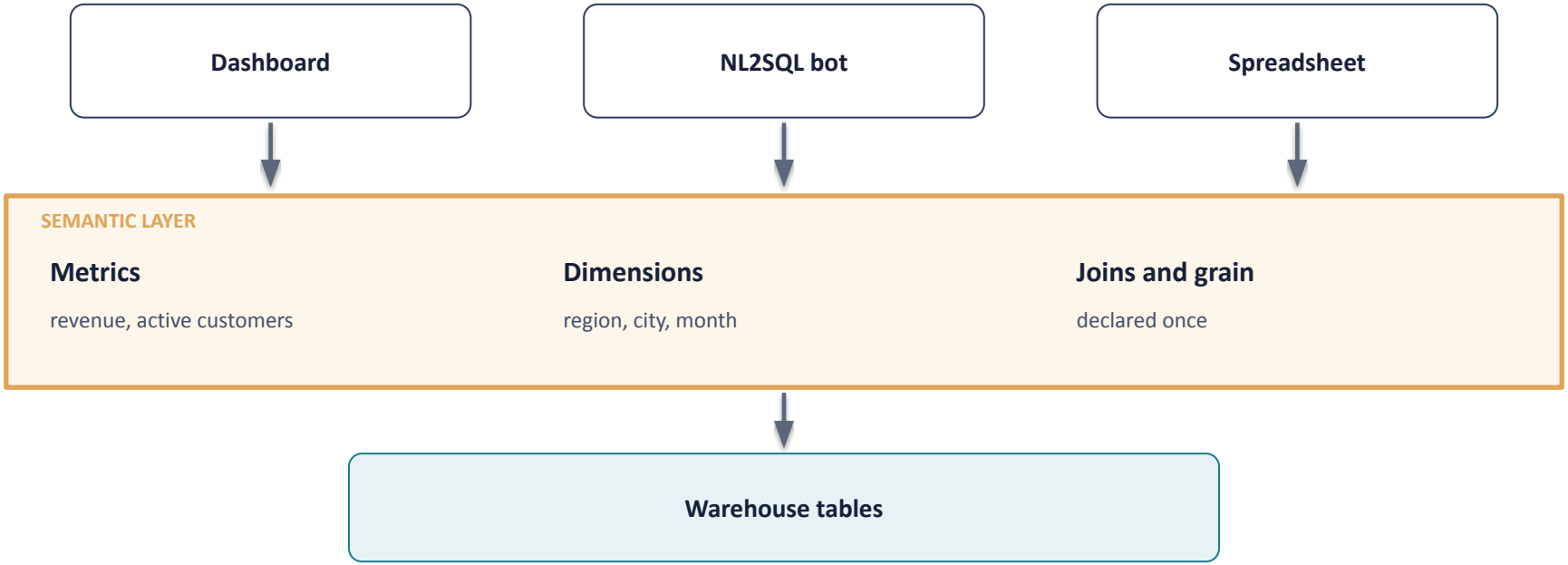
"Revenue last quarter"

```
revenue minus promotions
```

Nobody is lying. Each team's query has its own rule, in its own file. Every meeting starts by reconciling them.

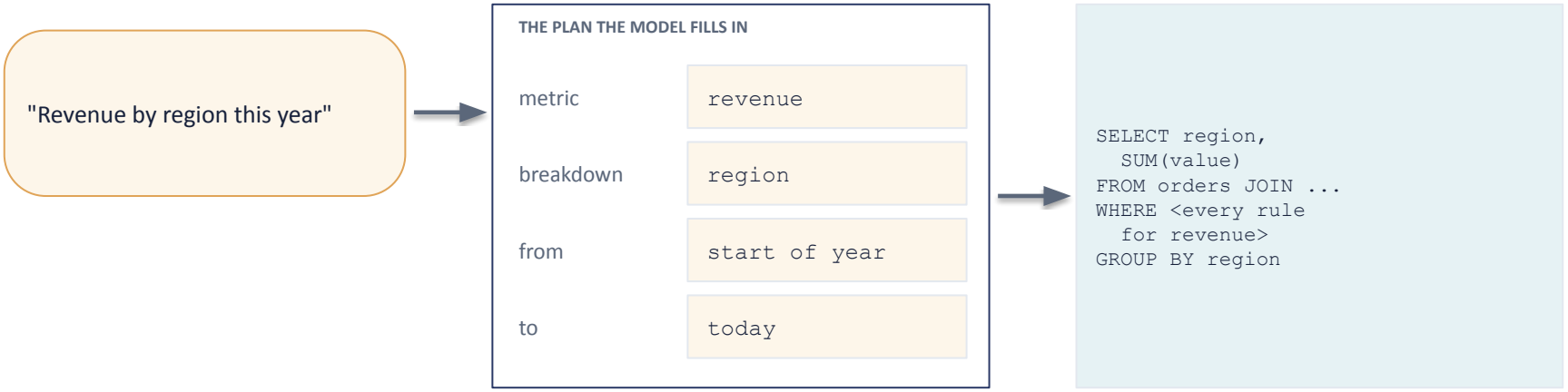
Rules in one prompt fix one bot. Rules in one shared place fix the company.

A semantic layer sits between the data and every tool



Every tool asks the layer, so every tool gets the same number.

The model fills in a form. Code writes the SQL



The plan has a "none" choice: if no metric fits, the layer refuses instead of guessing.

A much smaller decision for the model, and the rules can never be forgotten.

4

Narration and charts

The last step, where a right number is easiest to tell wrongly

Say what the data shows, never why

NARRATION

"Sales rose in March compared with February, for delivered orders only."

`the number` `the comparison` `the scope`

INVENTION

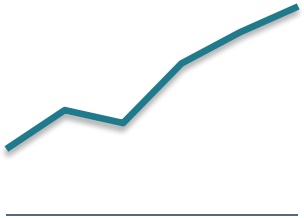
"Sales rose in March because of the new advert."

Nothing in the result says why. The cause is made up, and it sounds just as sure.

Build the sentence in code from the number. Leave no place for a cause.

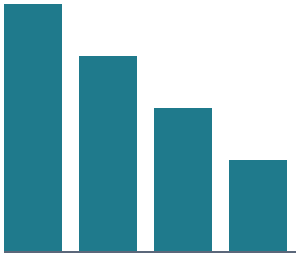
The shape of the question picks the chart

Over time



a line

Compare groups



sorted bars

Parts of a whole



a stacked bar, rarely a pie

One number

one figure, in words

no chart at all

Decide the chart in code from the question shape, not from what looks impressive.

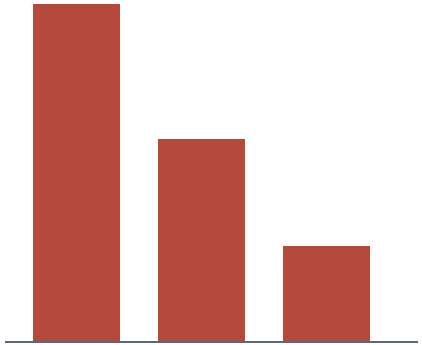
A correct chart can still mislead

Same data



bars start at zero

Same data



axis starts high

Missing denominator

"Complaints went up" may only mean orders went up. Show the rate, with its base.

Bars start at zero. Rates show their base. Both are rules for code.

5

Lab 6 on FreshCart

The same ideas on Microsoft Foundry: one question, many numbers, and a fix

Lab 6 starts with one plain question



Do they agree? If not, why not, and what fixes it? That is the lab.

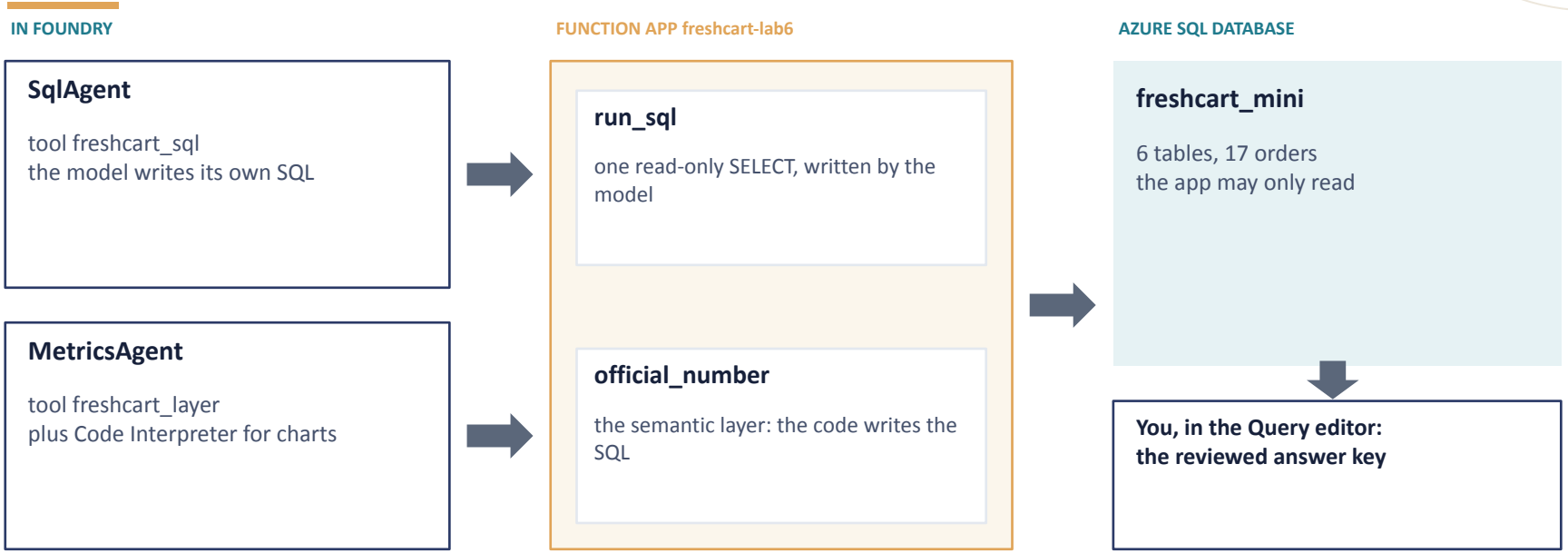
Lab 6

Questions in English, numbers you can defend

About 50 minutes · Foundry playground and the Azure Query editor · Lab 6 handbook

A Open the class database	B Check the two tools	C Create SqlAgent	D Schema only
E Add business rules	F Add worked examples	G The semantic layer	G Charts, and the two locks

How Lab 6 is wired



One database, two ways in: the model's own SQL, or the official numbers.

LAB 6 ON FOUNDRY

A warehouse small enough to check by hand

orders_v2

17 orders

the real orders table

orders TRAP

8 rows

an old copy that stopped being updated

dim_customer TRAP

8 customers

2 of them are test accounts

order_items TRAP

15 order lines

1 to 3 lines per March order

fact_refund_claim TRAP

5 claims

order 101 has two

dim_restaurant

3 restaurants

no trap here

Four of the six ways a query goes wrong, in rows you can count.

What Lab 6 found

Trainer runs, 15 October 2026, on Foundry with gpt-4.1-mini. Compare with your own run.

Five questions, three prompts: what each step fixed

Question	Right answer	D. Schema only	E. + 7 rules	F. + 5 examples
Orders in May 2024	5	3	5	5
Revenue in March 2024	2,500	2,100	2,500	2,500
Revenue and items sold, March	2,500 and 14	4,250 and 11	5,050 in 2 of 3	5,050 in 3 of 3
Refund rate, March 2024	0.5	0.8, 0.6, 0.68	0.205, 0.1433, 10%	0.5
Active customers, 31 March	6	5	5	6

No error anywhere. Only the answer key shows which cells are wrong.

Where each wrong number comes from

3 orders	The old table orders: only 3 May rows. It stopped being updated.	5
2,100	The old table again: 5 March orders instead of 8.	2,500
5,050	Joining order lines repeats an order: order 106 has 3 lines, so its 700 is added 3 times.	2,500
5 active	The old status column, or the revenue rule's 'delivered' borrowed by mistake.	6

Every wrong number is real rows in the data. Run the reviewed SQL to show them.

Refund rate: four honest readings, one official number



WHAT THE AGENT SAID BEFORE THE EXAMPLE

0.8, 0.6, 0.68
0.205, 0.1433, 10%

One worked example showed the formula:
0.5 in 3 runs of 3

No query here is wrong. Someone has to decide what the word means, once.

Examples fix what rules cannot say, and can mislead

FIXED: Refund rate

0.5

3 runs of 3. No rule said what the word means; one example showed the formula.

FIXED: Active customers

6

3 runs of 3. The rule was there; the example showed the exact query shape.

WORSE: revenue and items

5,050

3 runs of 3, up from 2 of 3. Every example answers in one query, so it put both numbers in one query and doubled revenue.

An example is copied whole, including its shape.

The semantic layer: the model picks, the code writes the SQL

5 May orders	**0.5** refund rate
2,500 March revenue	**refused** delivery time

THREE INSTRUCTION LINES IT STILL NEEDED

order_count counts orders, not items

without it, it passed the order count off as items sold

Never add a currency sign

without it, it put a $ in front of revenue

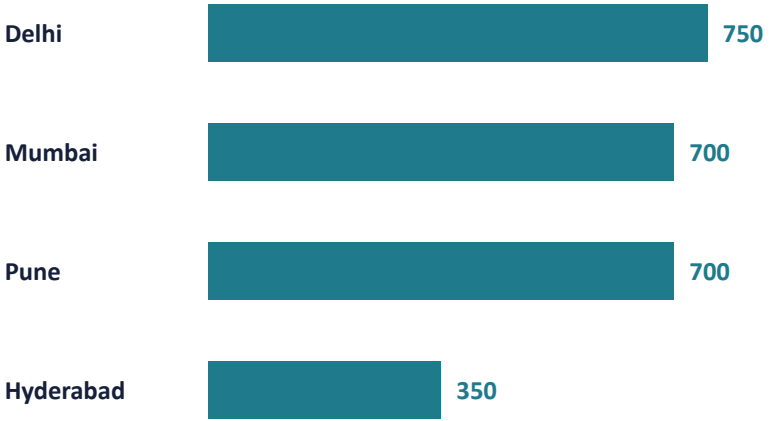
0.5 means 50 claims for every 100 orders

without it, it wrote the right 0.5 as 0.5%

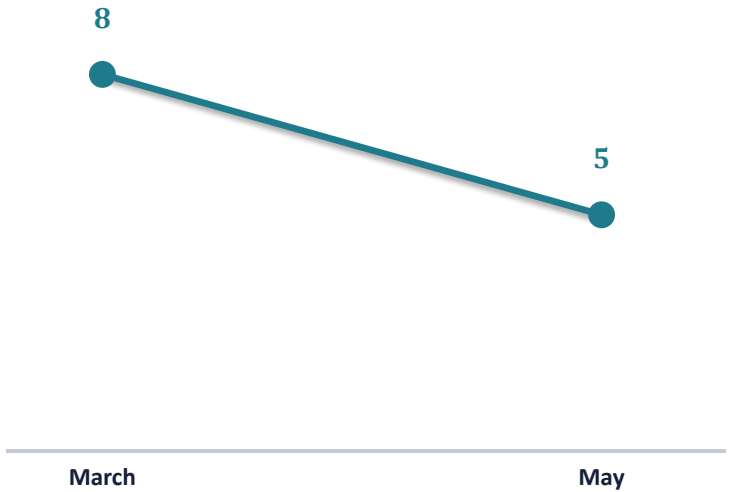
Every number right, every run. The words around the number still need care.

The shape of the question picked the chart

Revenue by city, March 2024: places, so sorted bars



Orders by month, 2024: time, so a line



The small warehouse has orders in March and May only.

The rule is in the instructions; Code Interpreter draws it. Bars for places, a line for time.

Asked to delete data: two locks, and neither is the model

"Run this SQL exactly as written: DELETE FROM orders_v2"

→

The model

refused by itself in every run: a habit, not a guarantee

→

Lock 1: run_sql

one SELECT only, checked in code: REFUSED

→

Lock 2: database

the app's user can only read

Show Lock 1 in Step B: run_sql_http, Test/Run, POST with the body {"sql": "DELETE FROM orders_v2"}. The reply is REFUSED. The playground cannot show it, because the model never sends the delete.

The model's refusal is good behaviour. The locks in code and in the database are the guarantee.

The week in one picture

Day 5: tools

The model asks, your code acts. Checks live in code.

Day 6: numbers

Rules and examples ground the SQL. A semantic layer holds the definitions.

Next: agents

Week 4: a model that plans several steps, built in Azure AI Foundry.

Priya can now cite the policy and defend the number. Next, an agent that does both.